

LECTUREFORGE

Nash Bruce V. Quiros

May 2026

LECTUREFORGE

A Project Paper

Presented to

James Bernard C. Febrio

Instructor of the Institute of Computer Studies

Tangub City Global College

In Partial Fulfillment of the Course

Multimedia Systems

Nash Bruce V. Quiros

May 2026

TABLE OF CONTENTS

Introduction.....2
How It Works.....2
Input, Process, and Output.....3
Simple Diagram.....3
Screenshots.....4
Testing.....7
Conclusion.....8

List of Figure and Tables

Table 1: Input, Process, and Output
Figure 1: System Flow
Figure 2 to 6: Screenshots of the App
Table 2: Test Case

Introduction

LectureForge helps students and teachers to cut the dead air out of lecture recordings in minutes every day.

A recording of a one-hour class is rarely one hour of teaching. Between people walking in, papers shuffling, long pauses while the teacher writes on the board, and the slow "record ... now it is recording" start of every session, 10 to 15 minutes of a lecture end up being silence. Cleaning that out by hand in a regular audio editor means zooming in, selecting, and deleting every single silent patch one at a time — a boring, slow job that nobody has time for.

Mostly students who review class recordings and want a shorter file to get through, and teachers who share their recordings with the class. Because the finished file is smaller, it also helps anyone with slow or metered internet — the condensed MP3 is much friendlier to download and send through messengers.

LectureForge does the boring part automatically. It listens to the recording, marks every silent gap, shows each cut directly on the waveform so the user can keep any gap they want (a laugh, a question from the class), speeds the audio up to 4× without turning the voice into a chipmunk, and exports a finished MP3 or WAV — all inside the browser. Nothing is ever uploaded anywhere: no server, no account, no waiting.

How It Works

The system follows a simple Input → Process → Output flow, shown below:

Input, Process, and Output

Table 1:

Input, Process, and Output

Stage	What happens
Input	A lecture recording the user picks (MP3, WAV, M4A, OGG, FLAC, or WebM) — or a live recording captured straight from the microphone after a short countdown.
Process	The file is decoded to raw audio on the computer itself. Loudness is measured in small frames to find the silent gaps; the user reviews every gap on the waveform and can keep any of them; the kept parts are stitched together sample by sample, optionally sped up with the pitch preserved, and encoded to MP3 or WAV.
Output	A condensed audio file saved straight to the device (example below: 10:27 became 5:05), plus an entry in the export history.

Simple Diagram

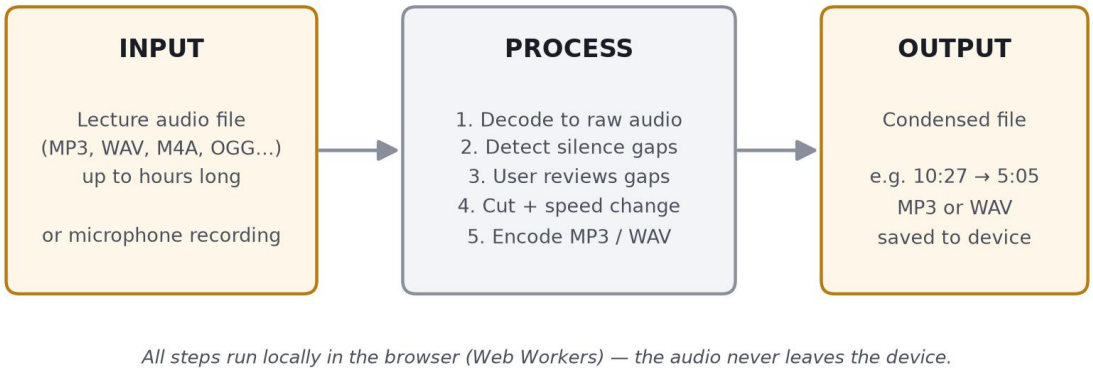


Figure 1:

System flow — input, process, and output of LectureForge.

Screenshots

The five screenshots below follow one complete session, using a 10-minute sample lecture that was recorded with 20 silent gaps inserted in it:

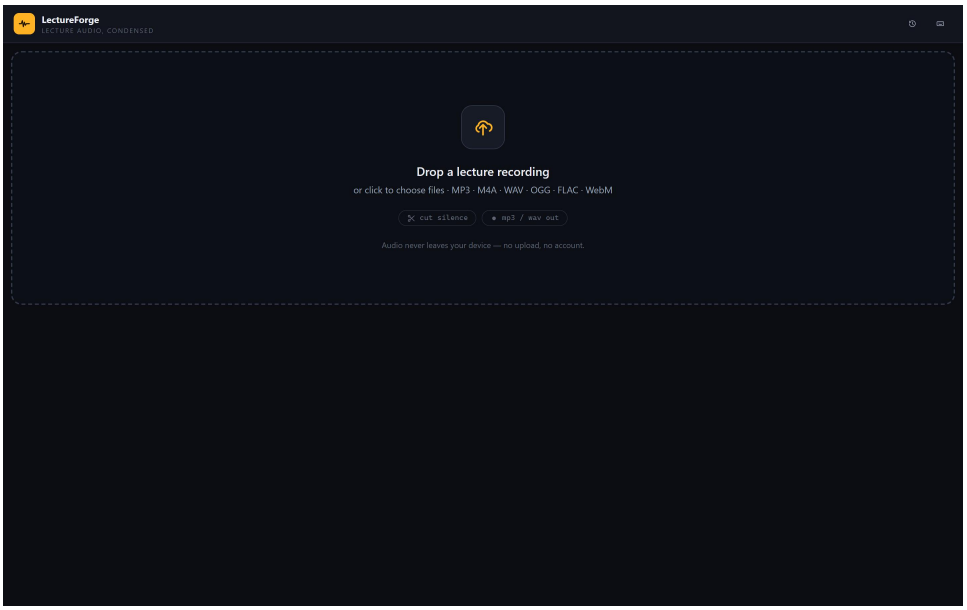


Figure 2. The main screen. The user drops a lecture recording into the window or clicks to choose a file. Nothing leaves the device.

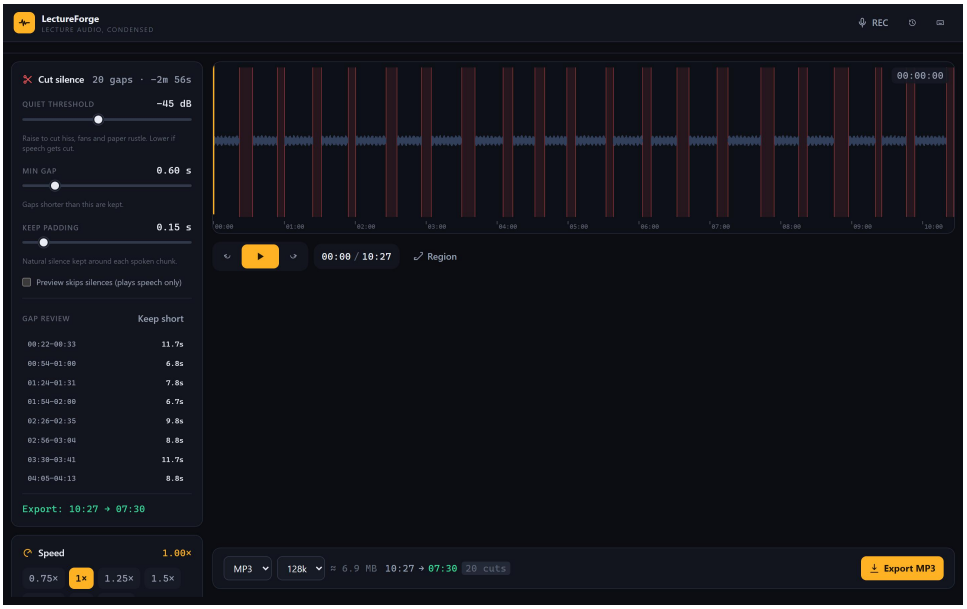


Figure 3. About two seconds later, the waveform appears with all 20 silent gaps marked in red. The side panel sums it up: "20 gaps · −2m 56s" of dead air.

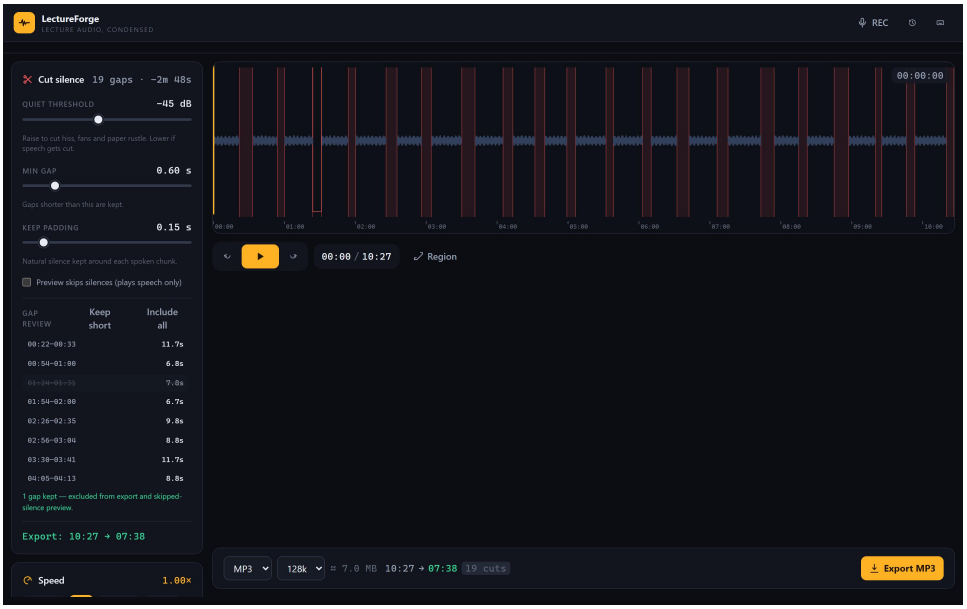


Figure 4. Gap review. Here the user kept one gap (a pause that was worth keeping) by clicking it — it now sits behind a strikethrough, and the export estimate updated itself from 07:30 to 07:38.

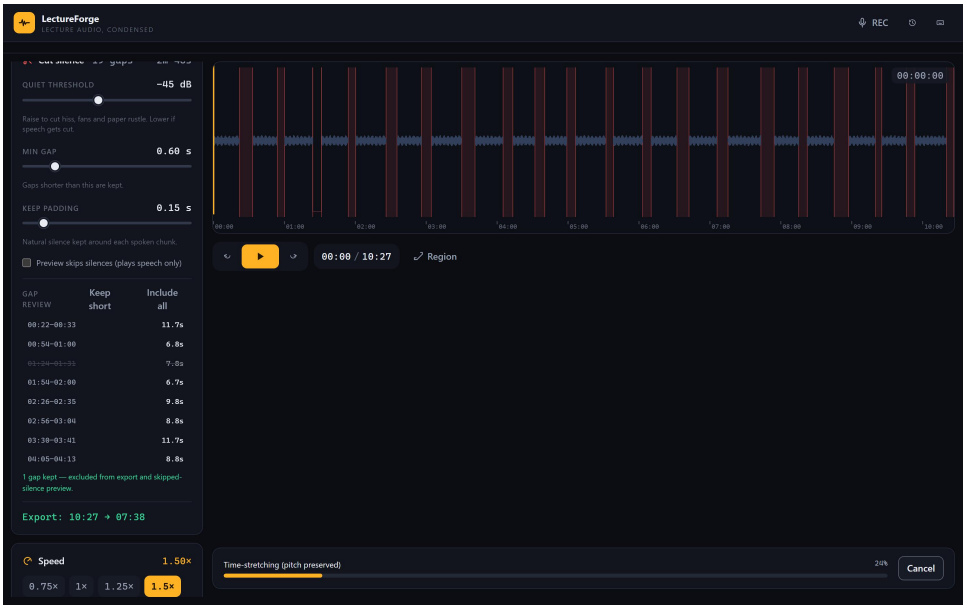


Figure 5. Exporting. Speed is set to 1.5×; the progress bar is the pitch-preserving time-stretch phase, which can be cancelled at any moment.

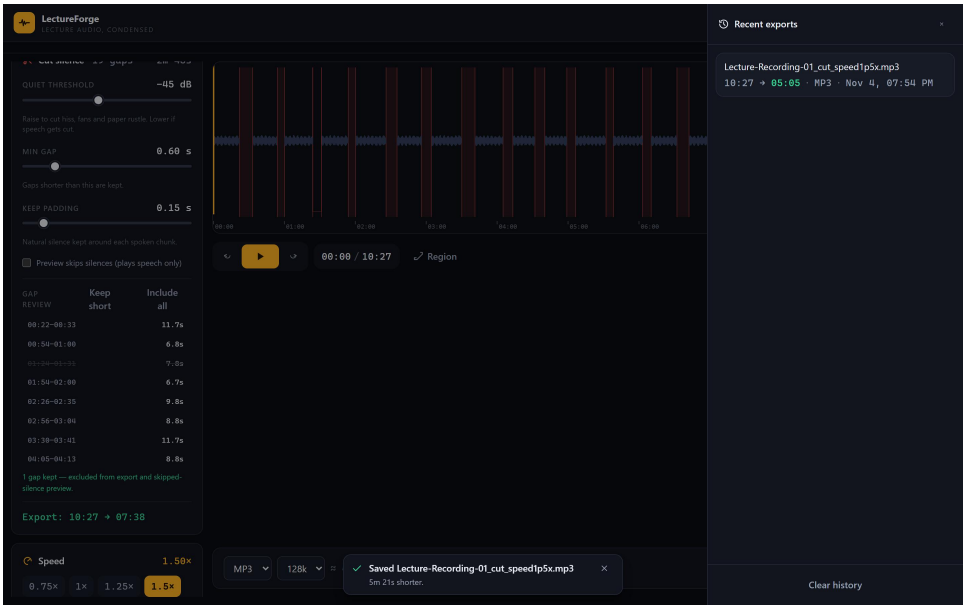


Figure 6. Finished. The history drawer remembers the new file ("10:27 → 05:05 · MP3") and a toast confirms it: 5 minutes 21 seconds shorter, in about 23 seconds of work.

Testing

Every test below was run on the same sample: a 10.4-minute lecture recording (105.5 MB) with 20 silent gaps inserted, about 3 minutes of dead air in total. Times were actually measured during the tests, not guessed.

Table 2:

Test Cases

Test	What you tried	Expected result	Actual result	Pass or Fail
Test 1	Load the 105.5 MB lecture file	Waveform and duration appear after a short analysis	Waveform appeared in 2.3 s, duration shown as 10:27	Pass
Test 2	Automatic silence detection on the sample lecture	All 20 inserted silent gaps are found and listed	20 of 20 gaps detected; 2m 56s of silence marked on the waveform	Pass
Test 3	Keep one gap from the gap review list	That gap stays in the export and the stats update	Stats changed to 19 cuts (10:27 → 07:38); the kept gap was drawn as a hollow outline	Pass
Test 4	Export MP3 at 1.5× speed with the cuts applied	Valid MP3 of about 5:05 with natural (not chipmunk) pitch	Export finished in 20.5 s; valid MP3; duration exactly 5:05; pitch stays natural	Pass

Test	What you tried	Expected result	Actual result	Pass or Fail
Test 5 (comparison)	Cut the silence from the same 10.4-minute lecture with LectureForge, then the same task the manual way in Audacity	LectureForge is faster and needs far fewer steps	LectureForge: 3 clicks and about 23 s total (2.3 s analysis + 20.5 s export). Manual way in Audacity: open the file, zoom in, then select and delete each of the 20 gaps one by one, then export — over 15 minutes and 60+ interactions for the same result	Pass

For the comparison (Test 5) the thing measured was simple: time taken, plus how many clicks or actions each way needs. On top of these manual tests, the audio engine behind the screen is covered by 28 automated tests (silence math, cutting, speed stretching, loudness leveling, and MP3/WAV encoding) — all 28 pass.

Conclusion

1. **What works:** the features that passed the tests: loading a lecture in about 2 seconds, finding all 20 silent gaps automatically, keeping gaps with one click, and exporting a finished MP3 at 1.5× in 20.5 s — condensing 10:27 down to 5:05 in roughly 23 seconds and 3 clicks. pitch-preserving speed-up (0.5× to 4×), loudness leveling for quiet recordings, batch queuing, an export history, a microphone recorder, and it all runs offline in the browser with no upload and no account. the 28 automated engine tests all pass, and an accessibility audit of the interface reported no serious issues.
2. **What does not work:** video is not supported — audio only; the microphone recorder saves in the browser's own format, which must be re-exported to MP3. files over 220 MB are analyzed at a lower sample rate, which slightly

softens the preview waveform, and older Safari versions are not supported. the Audacity comparison time is an honest estimate of the manual workflow, not a stopwatch recording of one person doing it.

3. **What I would improve:** extract audio from video files automatically, so lectures recorded as video can be tightened the same way. chapter markers in the exported MP3, so students can jump straight back to the questions they chose to keep. a Filipino-language interface for local classrooms, and better support for older iOS Safari versions.